

Les technologies de Suivi_eleve

Document d'apprentissage. Il explique **chaque technologie** utilisée dans le projet : ce que c'est, pourquoi on l'a choisie, **où** elle se trouve dans le code et un **petit exemple tiré du projet**. À la fin : un parcours conseillé pour apprendre ces technologies dans le bon ordre.

Les deux autres documents du dossier Help : [2-Structure-et-code](#) (le code expliqué fichier par fichier) et [3-Guide-utilisation](#) (comment se servir de l'application).

Sommaire

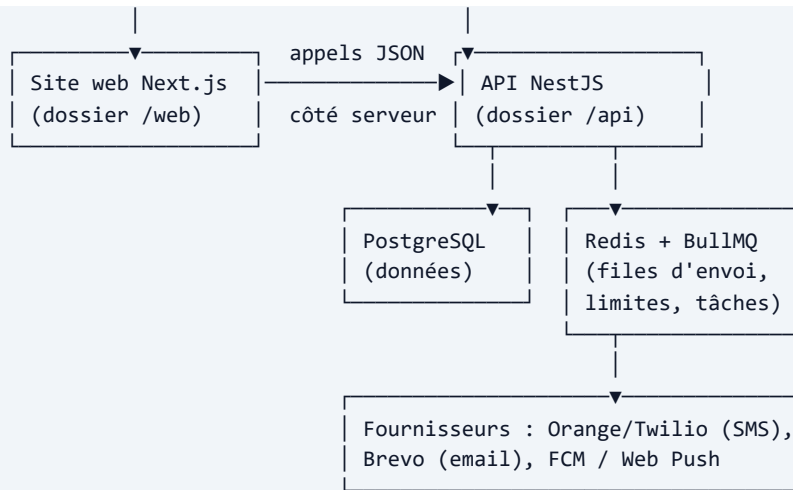
1. Vue d'ensemble : comment les morceaux se parlent
2. Les langages
3. Node.js et npm : le moteur et le gestionnaire de paquets
4. Le serveur (API) : NestJS
5. La base de données : PostgreSQL et Prisma
6. Les files d'attente : Redis et BullMQ
7. La sécurité : JWT, argon2, helmet, validation
8. Les envois : SMS, email, push
9. Les bibliothèques utilitaires de l'API
10. Le site web : React, Next.js et Tailwind CSS
11. Le site installable : PWA, service worker, Web Push
12. L'application mobile : React Native et Expo
13. La qualité : tests, lint, formatage, types
14. La mise en production : Docker, Caddy, GitHub Actions, EAS
15. Git et GitHub
16. Tableau récapitulatif des versions
17. Parcours d'apprentissage conseillé

1. Vue d'ensemble : comment les morceaux se parlent

Suivi_eleve est composé de **trois applications** qui partagent **une seule API** (le « cerveau ») :

PARENTS / PERSONNEL





- **L'API** (/api) contient toute la logique : qui a le droit de faire quoi, les élèves, les absences, l'envoi des messages... Elle parle **JSON** (un format de texte pour échanger des données).
- **Le site web** (/web) affiche les pages pour la direction, le personnel, les parents et le concepteur. Il demande les données à l'API **depuis le serveur** (le navigateur ne parle jamais directement à l'API).
- **L'application mobile** (/mobile) est installée sur les téléphones ; elle appelle l'API directement.
- **PostgreSQL** garde les données de façon durable ; **Redis** sert de mémoire rapide pour les files d'attente des envois.
- **Caddy** est la porte d'entrée sur Internet : il obtient le certificat HTTPS et répartit les requêtes.

Tout le code est dans **un seul dépôt Git** (un « monorepo ») géré par **pnpm**.

2. Les langages

2.1 TypeScript (le langage principal, ~95 % du code)

C'est quoi ? TypeScript est du **JavaScript avec des types**. On écrit le type de chaque donnée (texte, nombre, objet...) et un outil (le compilateur `tsc`) vérifie **avant l'exécution** qu'on ne mélange pas tout. Ensuite TypeScript est transformé en JavaScript, que Node.js ou le navigateur exécutent.

Pourquoi ? Une erreur comme « passer un nombre là où on attend un numéro de téléphone » est détectée tout de suite dans l'éditeur, pas chez un parent.

Exemple du projet (`api/src/auth/auth.types.ts`) :

```
export interface UtilisateurConnecte {  
  id: string; // texte  
  role: Role; // une des valeurs ADMIN, PARENT...  
  ecoleId: string;  
  tuteurId: string | null; // texte OU rien (null)  
}
```

Une `interface` décrit la **forme** d'un objet. Si on oublie `ecoleId`, TypeScript refuse de compiler.

Notions TypeScript utilisées dans le projet (toutes expliquées en détail dans le document 2) : `interface` , `type` , unions (`'clair' | 'sombre'`), génériques (`Page<T>`), `Record<Clé, Valeur>` , `as const` , `satisfies` , `readonly` , classes, décorateurs (`@Get()`), `async / await` , `import / export` .

Fichiers : tous les `.ts` et `.tsx` (le `x` signifie « contient du JSX », voir React).

2.2 JavaScript

TypeScript **devient** du JavaScript. Quelques fichiers sont écrits directement en JavaScript car ils sont exécutés tels quels par le navigateur ou un outil :

- `web/public/sw.js` : le service worker (programme qui tourne en arrière-plan dans le navigateur).
- `web/postcss.config.mjs` , `web/eslint.config.mjs` , `mobile/eslint.config.js` : fichiers de configuration (`.mjs` = module JavaScript moderne).

2.3 SQL

C'est quoi ? Le langage des bases de données relationnelles (`SELECT` , `INSERT` , `CREATE TABLE` ...).

Dans le projet, on écrit très peu de SQL à la main : **Prisma** le génère. On le voit dans `api/prisma/migrations/*/migration.sql` : chaque fichier contient les ordres SQL qui font évoluer la base (création de tables, ajout de colonnes). Exemple de nom : `20261005094554_init` = date + description.

2.4 CSS (avec Tailwind)

Le CSS décrit l'apparence (couleurs, tailles, marges). Le projet utilise surtout **Tailwind CSS** (voir §10.4) : on écrit des classes comme `rounded-lg px-4 text-sm` directement dans le HTML. Le seul gros fichier CSS est `web/app/globals.css` (couleurs « bleu école », mode sombre).

2.5 JSX / TSX

C'est quoi ? Une syntaxe qui permet d'écrire du « HTML » à l'intérieur de JavaScript/TypeScript. Utilisée par React (site web) et React Native (mobile).

```
<Titre>Bonjour {utilisateur.prenoms}</Titre>
```

Les accolades `{ }` insèrent une valeur JavaScript dans l'affichage.

2.6 Prisma Schema Language

Un petit langage propre à Prisma pour décrire les tables : `api/prisma/schema.prisma` . Exemple :

```
model Classe {
  id      String @id @default(uuid(7))
  nom     String // ex. 6e A
  niveau String // ex. 6e
  eleves  Eleve[]
}
```

2.7 YAML

Format de configuration lisible (indentation = hiérarchie). Fichiers : `docker-compose.yml` , `docker-compose.prod.yml` , `pnpm-workspace.yaml` , `.github/workflows/ci.yml` .

2.8 JSON

Format de données `{"cle": "valeur"}` . Sert à la fois pour **échanger** avec l'API et pour **configurer** : `package.json` , `tsconfig.json` , `app.json` , `eas.json` , `.prettierrc` ...

2.9 Shell (sh / bash)

Scripts de commandes Linux pour le serveur : `deploy/mettre-a-jour.sh` (mise à jour en une commande), `deploy/sauvegarde/sauvegarde.sh` et `restaurer.sh` (sauvegardes chiffrées).

2.10 Dockerfile et Caddyfile

Deux petits langages de configuration : les `Dockerfile` décrivent comment fabriquer une image (voir §14), le `Caddyfile` configure le serveur web Caddy.

2.11 Markdown

Le format de ces documents (`.md`) : `#` pour un titre, `**gras**` , listes avec `-` . Tous les documents du projet (`docs/` , `README.md` , `CLAUDE.md` , `Help/`) sont en Markdown.

3. Node.js et pnpm : le moteur et le gestionnaire de paquets

3.1 Node.js (version 22 ou plus)

C'est quoi ? Un programme qui exécute du JavaScript **en dehors du navigateur**, sur un serveur ou un ordinateur. L'API et le serveur du site Next.js tournent sur Node.js.

Ce qu'on utilise de Node.js (modules intégrés, importés avec le préfixe `node:`) :

- `node:crypto` : nombres aléatoires sûrs (`randomInt` pour les codes SMS), empreintes SHA-256 (`createHash`), identifiants uniques (`randomUUID`).
- `node:fs/promises` : lire/écrire des fichiers (photos dans `StockageService`).
- `node:path` : construire des chemins de fichiers.
- `node:async_hooks` (`AsyncLocalStorage`) : retenir l'IP de la requête en cours pour le journal d'audit.
- `node:util` (`parseArgs`) : lire les options de la ligne de commande (`initialiser.ts`).

Modules ES : le projet utilise la syntaxe moderne `import ... from '...'` (et non l'ancienne `require`). Dans l'API, les imports locaux finissent par `.js` même si le fichier s'appelle `.ts` : c'est la règle de Node.js pour les modules ES (le fichier compilé sera bien un `.js`).

3.2 npm, les paquets et `package.json`

Un **paquet** est une bibliothèque de code partagée (ex. `bullmq`, `next`). On les télécharge depuis le registre **npm**. Chaque application a un `package.json` qui liste :

- `dependencies` : paquets nécessaires pour fonctionner ;
- `devDependencies` : paquets utiles seulement pour développer (tests, lint...) ;
- `scripts` : commandes raccourcies (`pnpm test`, `pnpm build` ...).

Le symbole `^12.0.1` veut dire « version 12.0.1 ou plus récente, mais toujours 12.x ». `~57.0.26` veut dire « 57.0.x ».

3.3 pnpm et le monorepo

pnpm remplace `npm` : il est plus rapide et économise le disque. Le fichier `pnpm-workspace.yaml` déclare **trois projets** dans le même dépôt :

```
packages:
- api
- web
- mobile
```

`pnpm-lock.yaml` fige les versions **exactes** de tous les paquets (pour que tout le monde installe la même chose).

Commandes utiles (depuis la racine) :

Commande	Effet
<code>pnpm install</code>	installe les paquets des trois projets
<code>pnpm dev:api</code> / <code>pnpm dev:web</code> / <code>pnpm dev:mobile</code>	lance une application en mode développement
<code>pnpm lint</code> / <code>pnpm typecheck</code> / <code>pnpm test</code> / <code>pnpm format:check</code>	les 4 vérifications obligatoires
<code>pnpm db:migrate</code> / <code>pnpm db:seed</code>	met à jour la base / la remplit de données de test
<code>pnpm --filter @suivi-eleve/api <commande></code>	lance une commande dans un seul projet

4. Le serveur (API) : NestJS

Version : NestJS 12. Dossier : `/api/src` .

4.1 C'est quoi une API ?

Une **API REST** est un programme qui répond à des **requêtes HTTP**. Chaque requête a :

- une **méthode** : `GET` (lire), `POST` (créer / agir), `PATCH` / `PUT` (modifier), `DELETE` (supprimer) ;
- un **chemin** : `/api/eleves/123` ;

- éventuellement un **corps** JSON (les données envoyées).

La réponse a un **code** : 200 OK, 201 créé, 204 sans contenu, 400 données invalides, 401 non connecté, 403 interdit, 404 introuvable, 429 trop de tentatives, 500 erreur du serveur.

4.2 Pourquoi NestJS ?

NestJS est un **cadre** (framework) qui impose une organisation claire, idéale pour une grosse application : chaque fonctionnalité est un **module** qui contient un **contrôleur** (les routes) et un **service** (la logique). Il est écrit pour TypeScript et utilise beaucoup les **décorateurs** (`@Get()` , `@Roles()` ...). Il s'appuie sur **Express**, le serveur HTTP le plus connu de Node.js.

4.3 Les briques de NestJS (avec exemples du projet)

Brique	Rôle	Exemple dans le projet
Module (<code>@Module</code>)	Regroupe une fonctionnalité	<code>annonces.module.ts</code>
Contrôleur (<code>@Controller</code>)	Reçoit les requêtes HTTP, appelle le service	<code>annonces.controller.ts</code> : <code>@Post()</code> crée une annonce
Service (<code>@Injectable</code>)	La logique métier, les accès à la base	<code>annonces.service.ts</code>
DTO (Data Transfer Object)	La forme des données reçues, avec leurs règles de validation	<code>annonces.dto.ts</code> : <code>CreerAnnonceDto</code>
Guard (garde)	Décide si une requête a le droit de passer	<code>JwtAuthGuard</code> , <code>RolesGuard</code> , <code>ParentOwnsEleveGuard</code>
Pipe	Transforme / valide un paramètre	<code>ValidationPipe</code> (global), <code>ParseUUIDPipe</code>
Décorateur personnalisé	Ajoute une information à une route	<code>@Public()</code> , <code>@Roles(...)</code> , <code>@UtilisateurCourant()</code>
Cycle de vie	Code lancé au démarrage / à l'arrêt	<code>onModuleInit()</code> démarre les files BullMQ

4.4 L'injection de dépendances

C'est l'**idée centrale** de NestJS. Un service ne crée pas lui-même les objets dont il a besoin : il les **demande dans son constructeur**, et NestJS les lui donne.

```
@Injectable()
export class AnnoncesService {
  constructor(
    private readonly prisma: PrismaService, // accès à la base
    private readonly notifications: NotificationsService, // envoi des messages
    private readonly audit: AuditService, // journal
  ) {}
}
```

Avantages : un seul `PrismaService` pour toute l'application, et dans les tests on peut remplacer un service par un faux.

4.5 Le module de configuration

@nestjs/config lit le fichier `.env` (variables secrètes : mot de passe de la base, clés...). On lit une valeur avec `config.get('REDIS_URL')`. Le fichier `.env.example` liste toutes les variables possibles.

4.6 Documentation des routes

Toutes les routes sont sous le préfixe `/api` (ex. `GET /api/sante` répond `{"statut":"ok"}`). La liste complète des routes est contrôlée automatiquement par le test `api/test/securite.e2e-spec.ts`.

5. La base de données : PostgreSQL et Prisma

5.1 PostgreSQL 17

C'est quoi ? Un **système de base de données relationnelle** libre et très fiable. Les données sont rangées dans des **tables** (comme des feuilles Excel) reliées entre elles :

- une table `eleves` (une ligne par élève),
- une table `tuteurs`,
- une table de liaison `eleves_tuteurs` (quel tuteur pour quel élève).

Notions importantes :

- **Clé primaire** (`id`) : identifiant unique d'une ligne. Ici des **UUID v7** (ex. `0192f3a4-...`), uniques au monde et triables par date.
- **Clé étrangère** : une colonne qui pointe vers une autre table (`eleves.classe_id` → `classes.id`).
- **Index** : accélère les recherches (`@@index([ecoleId, statut])`).
- **Contrainte d'unicité** : interdit les doublons (`@@unique([ecoleId, contact1])`) : un même numéro ne peut pas être deux tuteurs différents dans la même école).
- **Transaction** : plusieurs opérations qui réussissent **toutes** ou **aucune**.

En développement, PostgreSQL tourne dans Docker (`docker-compose.yml`, port 5432).

5.2 Prisma 7 (l'ORM)

C'est quoi ? Un **ORM** (Object-Relational Mapping) : il permet de lire et écrire dans la base **en TypeScript**, sans écrire de SQL, avec des types vérifiés.

Trois éléments :

1. **Le schéma** `api/prisma/schema.prisma` : la description de toutes les tables (24 modèles : `Ecole`, `Utilisateur`, `Eleve`, `Tuteur`, `Appareil`, `Notification` ...) et des listes de valeurs (`enum Role { ADMIN SECRETARIAT ... }`).
2. **Les migrations** `api/prisma/migrations/` : à chaque changement du schéma, `pnpm db:migrate` crée un fichier SQL daté et l'applique. **Règle du projet : ne jamais modifier une migration déjà poussée.** En production, `prisma migrate deploy` applique les migrations en attente au démarrage de l'API.

3. **Le client généré** `api/src/generated/prisma` : du code TypeScript fabriqué à partir du schéma (`pnpm db:generate`). Il n'est pas versionné dans Git.

Exemple de requête Prisma (tiré de `annonces.service.ts`) :

```
const annonce = await this.prisma.annonce.findFirst({
  where: { id, ecoleId: u.ecoleId }, // WHERE id = ... AND ecole_id = ...
  select: { statut: true }, // ne lire que la colonne statut
});
```

Prisma traduit cela en SQL. Autres méthodes vues dans le projet : `findMany`, `findUnique`, `create`, `createManyAndReturn`, `update`, `updateMany`, `upsert` (créer ou mettre à jour), `count`, `groupBy` (statistiques), `$transaction([...])`.

L'adaptateur `@prisma/adapters-pg` relie Prisma au pilote PostgreSQL `pg` (nouveau de Prisma 7). Voir `api/src/prisma/prisma.service.ts`.

Le seed `api/prisma/seed.ts` remplit la base de développement avec une école fictive, 2 classes, 20 élèves, 25 tuteurs (`pnpm db:seed`).

Prisma Studio (`pnpm db:studio`) ouvre une interface web pour regarder les tables.

6. Les files d'attente : Redis et BullMQ

6.1 Redis 8

C'est quoi ? Une base de données **en mémoire**, ultra-rapide, qui stocke des clés et des valeurs. Dans le projet, Redis sert à :

- stocker les **files d'attente** des envois (via BullMQ) ;
- compter les **tentatives de connexion** (`LimiteurService` : au-delà de 20 essais en 15 minutes, l'IP est bloquée).

En production, Redis est lancé avec `--appendonly yes` : il écrit aussi sur disque pour ne rien perdre au redémarrage.

6.2 BullMQ 6

C'est quoi ? Une bibliothèque de **files de tâches** construite sur Redis. Au lieu d'envoyer 500 SMS pendant que la secrétaire attend, l'API **dépose 500 tâches** dans une file et répond tout de suite ; des **travailleurs** (workers) les traitent en arrière-plan.

Concepts :

- **Queue** (file) : là où on ajoute les tâches (`file.add('envoi', { notificationId })`).
- **Worker** (travailleur) : la fonction qui traite chaque tâche.
- **Essais et délai croissant** (`attempts: 3, backoff: exponential`) : si le fournisseur SMS ne répond pas, on réessaie après 30 s, puis 60 s...

- **Priorité** : une libération anticipée (URGENTE = 1) passe avant un reçu (BASSE = 4).
- **Tâche différée** (`delay`) : une annonce programmée pour 18 h est mise en file avec un délai.
- **Tâche répétée** (`upsertJobScheduler` avec un motif `cron`) : les relances de paiement automatiques tous les jours à 9 h (`'0 9 * * *'`).

Files du projet : `notifications-sms`, `notifications-email`, `notifications-push` (dans `envoi.service.ts`), `annonces` (envois programmés et rappels de la veille), `paiements` (relances quotidiennes).

7. La sécurité : JWT, argon2, helmet, validation

7.1 JWT (JSON Web Token) — `@nestjs/jwt`

C'est quoi ? Un « badge » numérique signé par le serveur. Il contient l'identifiant, le rôle et l'école de l'utilisateur. Le serveur vérifie la **signature** (avec `JWT_SECRET`) à chaque requête : impossible de fabriquer un faux badge.

Dans le projet :

- **Jeton d'accès** : JWT valable **15 minutes**, envoyé dans l'en-tête `Authorization: Bearer <jeton>`.
- **Jeton de rafraîchissement** : valable **30 jours**, sert à obtenir un nouveau jeton d'accès. Seule son **empreinte SHA-256** est stockée en base. Chaque utilisation le remplace (rotation) ; si un ancien jeton revient, toutes les sessions de l'utilisateur sont fermées (protection contre le vol).

7.2 argon2 — `@node-rs/argon2`

C'est quoi ? Un algorithme de **hachage de mot de passe** (gagnant d'un concours international). On ne stocke jamais un mot de passe, seulement son « hachage » : une empreinte impossible à inverser et volontairement lente à calculer (pour décourager les pirates). Les codes SMS à 6 chiffres sont aussi hachés.

7.3 Helmet

Ajoute des **en-têtes de sécurité HTTP** aux réponses de l'API (interdire l'affichage dans une iframe, etc.). Voir `app.setup.ts`. Le site web a ses propres en-têtes (CSP) dans `next.config.ts`.

7.4 class-validator et class-transformer

class-validator vérifie les données reçues grâce à des décorateurs posés sur les DTO :

```
@Matches(/^\\d{6}$/, { message: 'Le code contient 6 chiffres.' })
code: string;
```

class-transformer les transforme avant la vérification (ex. enlever les espaces d'un numéro de téléphone, convertir `"2"` en nombre `2`).

Le `ValidationPipe` global (dans `app.setup.ts`) applique ces règles à **toutes** les routes et refuse les champs inconnus (`forbidNonWhitelisted`).

7.5 Limitation des tentatives

`LimiteurService` (Redis) bloque les attaques par force brute : 20 connexions par IP / 15 min, 10 par compte, 10 demandes de code SMS par IP / heure...

7.6 Le contrôle d'accès

- `JwtAuthGuard` : toute route exige un jeton, sauf `@Public()`.
- `RolesGuard` : `@Roles(Role.ADMIN)` réserve une route à certains rôles ; le concepteur n'entre jamais dans les données des écoles.
- `ParentOwnsEleveGuard` : un parent ne voit que **ses** enfants.
- Chaque requête du personnel est filtrée par `ecoleId` (une école ne voit jamais une autre école).
- `AuditService` journalise les actions sensibles (avec l'IP).

8. Les envois : SMS, email, push

Le **moteur de notifications** (`api/src/notifications`) choisit le fournisseur selon la configuration. Les modules métier n'appellent **jamais** un fournisseur directement : ils passent par `NotificationsService.notifier(...)`.

Canal	Fournisseurs possibles	Variable
SMS	Orange SMS API, Twilio (et un fournisseur de secours)	<code>SMS_FOURNISSEUR</code> , <code>SMS_FOURNISSEUR_SECOURS</code>
Email	Brevo	<code>EMAIL_FOURNISSEUR</code>
Push mobile	FCM (Firebase Cloud Messaging de Google)	<code>PUSH_FOURNISSEUR</code>
Push du site	Web Push (standard des navigateurs, clés VAPID)	<code>VAPID_*</code>
Essai / tests	<code>console</code> (écrit dans les logs), <code>simulation</code> (tests)	<code>ENVOIS_SIMULES=oui</code>

Ces services sont appelés avec `fetch` (la fonction standard pour faire une requête HTTP) dans `fournisseurs.ts`. Les **webhooks** (`/api/notifications/webhooks/...`) permettent aux fournisseurs de signaler qu'un SMS a été **délivré**.

L'alphabet GSM : un SMS fait 160 caractères seulement s'il n'utilise que l'alphabet GSM ; un seul `ê` ou `'` le fait passer à 70 caractères. La fonction `versGsm7` remplace donc ces caractères.

9. Les bibliothèques utilitaires de l'API

Paquet	Usage dans le projet
<code>pdfkit</code>	Fabrique les bulletins PDF (<code>resultats/bulletin.ts</code>) et les planches d'étiquettes (<code>appareils/etiquettes.ts</code>)
<code>qrcode</code>	Dessine les QR codes collés sur les appareils des élèves
<code>exceljs</code>	Lit et écrit les fichiers Excel (import / export des élèves)

Paquet	Usage dans le projet
web-push	Envoie les notifications Web Push chiffrées et génère les clés VAPID
ioredis	Client Redis (limiteur de tentatives)
rxjs	Programmation réactive, utilisée en interne par NestJS
reflect-metadata	Permet aux décorateurs de stocker des informations (nécessaire à NestJS)
dotenv	Lit le fichier .env dans les scripts hors NestJS (seed.ts , configuration Prisma)
tsx	Exécute directement un fichier TypeScript (le seed)

10. Le site web : React, Next.js et Tailwind CSS

Dossier : /web . Port de développement : **3001**.

10.1 React 19

C'est quoi ? Une bibliothèque pour construire des interfaces à partir de **composants** : des fonctions qui renvoient de l'affichage (JSX). Un composant peut recevoir des **props** (paramètres) et avoir un **état** (useState) qui, quand il change, redessine l'écran.

```
export function ChoixTheme({ initial }: { initial: Theme }) {
  const [theme, setTheme] = useState(initial); // état
  ...
  return <button onClick={basculer}>{theme === 'sombre' ? '☀️' : '🌙'}</button>;
}
```

Hooks utilisés (fonctions qui commencent par use) : useState (état), useEffect (action après l'affichage), useRef (référence à un élément), useMemo / useCallback (mémoriser un calcul), useContext (données partagées), useActionState (résultat d'une Server Action), cache (lire une donnée une seule fois par requête).

10.2 Next.js 16 (App Router)

C'est quoi ? Un cadre au-dessus de React qui ajoute : le **routing par dossiers**, le **rendu côté serveur**, les **Server Actions**, l'optimisation des images et polices, la construction pour la production.

Attention : Next.js 16 diffère des versions précédentes. Le fichier web/AGENTS.md rappelle de lire la documentation embarquée web/node_modules/next/dist/docs/.

Le routing par dossiers (web/app) : chaque dossier est une partie de l'adresse, et page.tsx est la page affichée.

Fichier	Adresse
app/connexion/page.tsx	/connexion

Fichier	Adresse
app/(admin)/eleves/page.tsx	/eleves
app/(admin)/eleves/[id]/page.tsx	/eleves/123 ([id] = partie variable)
app/(parent)/parent/(espace)/messages/page.tsx	/parent/messages

Un dossier entre **parenthèses** (admin) est un **groupe** : il n'apparaît pas dans l'adresse, mais il permet de partager un `layout.tsx` (mise en page commune : bandeau + menu).

Fichiers spéciaux : `layout.tsx` (mise en page), `page.tsx` (page), `actions.ts` (Server Actions), `route.ts` (route qui renvoie un fichier, ex. téléchargement Excel), `manifest.ts` (manifeste PWA), `icon.svg`.

Composants serveur et composants client :

- Par défaut, un composant est **serveur** : il s'exécute sur le serveur, peut lire l'API (`await lireApi(...)`) et envoie au navigateur du HTML déjà prêt. Rapide et sûr (les jetons restent sur le serveur).
- Un fichier qui commence par `'use client'` est un **composant client** : il s'exécute dans le navigateur et peut réagir aux clics (`useState` , `onClick`). Exemples : `navigation.tsx` , `choix-theme.tsx` , `formulaire-action.tsx` .

Server Actions : des fonctions marquées `'use server'` (fichiers `actions.ts`) appelées directement par un formulaire. Elles s'exécutent sur le serveur, appellent l'API (`envoyerApi`), puis `refresh()` (relire la page) ou `redirect()` (changer de page).

`proxy.ts` (nouveau Next 16, remplace l'ancien `middleware`) : s'exécute avant chaque page ; il renouvelle le jeton d'accès quand il va expirer et renvoie vers `/connexion` si la session est finie.

`params` et `searchParams` **asynchrones** : dans Next 16, on écrit `(await props.searchParams).page` .

Cookies httpOnly : les jetons sont gardés dans des cookies que le JavaScript du navigateur **ne peut pas lire** (protection contre le vol de session).

Mode standalone : `next build` produit un petit serveur autonome pour l'image Docker.

10.3 ESLint

Vérifie le code du site et signale les erreurs probables (`eslint-config-next`). Commande : `pnpm lint` .

10.4 Tailwind CSS 4

C'est quoi ? Un CSS « utilitaire » : au lieu d'écrire une feuille de style, on compose des petites classes.

```
<button className="rounded-lg px-4 py-2 text-sm font-semibold text-white">
```

`rounded-lg` = coins arrondis, `px-4` = marge intérieure horizontale, `text-sm` = petit texte. Les préfixes `sm` , `md` : , `lg` : appliquent un style à partir d'une certaine largeur d'écran (site adapté aux téléphones). `hover:` = au survol.

Les couleurs du projet sont définies dans `web/app/globals.css` avec `@theme` : `marque-50` à `marque-950` (bleu école), `soleil-*` (jaune), `carte` (fond des cartes). **Règles** : utiliser `bg-carte` (jamais `bg-white`), pas de

classes `dark` ; le **mode sombre** est obtenu en **inversant les nuances** quand `<html data-theme="sombre">` .

PostCSS (`postcss.config.mjs`) : l'outil qui transforme le CSS ; Tailwind 4 s'y branche.

Polices : `next/font/google` charge Geist et Geist Mono.

11. Le site installable : PWA, service worker, Web Push

11.1 PWA (Progressive Web App)

Un site qu'on peut **installer** comme une application (icône sur l'écran d'accueil, plein écran), sur PC, Android et iPhone, sans passer par un magasin d'applications. Il faut :

- un **manifeste** (`web/app/manifest.ts`) : nom, couleurs, icônes (`public/icones`) ;
- un **service worker** (`web/public/sw.js`) ;
- le HTTPS.

Le bandeau `components/installation.tsx` propose l'installation (sur iPhone : « Partager » puis « Sur l'écran d'accueil »).

11.2 Le service worker

Un programme JavaScript qui tourne **en arrière-plan** dans le navigateur, même quand le site est fermé. Celui du projet :

- affiche une **page hors ligne** (`public/hors-ligne.html`) quand il n'y a pas de réseau ;
- reçoit les **notifications push** et les affiche ;
- ouvre le bon message quand on touche la notification ;
- **ne met jamais en cache** les données des élèves (vie privée).

11.3 Web Push et VAPID

Le **Web Push** est le standard qui permet à un serveur d'envoyer une notification à un navigateur. Le navigateur s'abonne (`pushManager.subscribe`) et fournit une adresse (endpoint) + des clés de chiffrement, enregistrées dans la table `jetons_push` (plateforme WEB). Les clés **VAPID** (`node dist/cli/cles-vapid.js`) identifient notre serveur auprès des services push (Google, Mozilla, Apple). Sur iPhone, cela ne marche que pour le site **ajouté à l'écran d'accueil**.

12. L'application mobile : React Native et Expo

Dossier : `/mobile` .

12.1 React Native 0.86

C'est quoi ? React, mais pour fabriquer de **vraies applications** Android et iPhone. Au lieu de `<div>` et `<p>`, on utilise des composants natifs : `<View>` (boîte), `<Text>` (texte), `<Pressable>` (zone touchable), `<TextInput>` (champ), `<ScrollView>`, `<ActivityIndicator>` (roue de chargement). Les styles s'écrivent en JavaScript avec `StyleSheet.create({...})` (proches du CSS : `flexDirection`, `padding` ...).

`react-native-web` permet aussi d'ouvrir l'application dans un navigateur (pratique pour tester sur iPhone sans compte Apple).

12.2 Expo SDK 57

C'est quoi ? Une boîte à outils au-dessus de React Native qui simplifie tout : lancement (`expo start`), accès au matériel par des modules prêts à l'emploi, construction des applications dans le cloud (EAS).

Règle du projet : ajouter un module avec `npm install <paquet>` (choisit la version compatible), jamais `pnpm add`.

Modules Expo utilisés :

Module	Usage
<code>expo-router</code>	Navigation par dossiers (comme Next.js) dans <code>src/app</code>
<code>expo-secure-store</code>	Garde le jeton de session dans le trousseau chiffré du téléphone
<code>@react-native-async-storage/async-storage</code>	Petite mémoire non chiffrée : thème choisi, derniers messages (lecture hors réseau)
<code>expo-notifications</code>	Permission, jeton push, affichage et clic des notifications
<code>expo-camera</code>	Lecture des QR codes des appareils (surveillants)
<code>expo-file-system</code> , <code>expo-sharing</code>	Télécharger et partager un bulletin PDF
<code>expo-device</code> , <code>expo-constants</code>	Savoir si on est sur un vrai téléphone, dans Expo Go...
<code>expo-linking</code> , <code>expo-status-bar</code> , <code>react-native-screens</code> , <code>react-native-safe-area-context</code>	Liens, barre d'état, écrans natifs, zones sûres (encoche)
<code>expo-dev-client</code>	Application de développement personnalisée

Expo Go : application gratuite pour tester le projet sur un téléphone Android en scannant un QR code (`exp://...:8090`).

12.3 Expo Router

Comme Next.js : les fichiers de `mobile/src/app` sont des écrans.

- `(parent)/(onglets)/index.tsx` → onglet Accueil du parent ;
- `(parent)/notification/[id].tsx` → détail d'un message ;

- `(personnel)/scanner.tsx` → scanner de QR code.

`<Stack>` empile les écrans (bouton retour), `<Tabs>` affiche la barre d'onglets en bas, `<Stack.Protected guard={...}>` n'autorise un groupe d'écrans qu'au bon profil.

12.4 Thème et couleurs

`src/lib/theme.tsx` définit deux palettes (`CLAIR` , `SOMBRE`) et un **contexte React** qui les partage. **Règle** : jamais de couleur en dur, toujours `useTheme()` ou `useStyles(creerStyles)` ; les dégradés via `degrade()` .

12.5 Fichiers `.web.ts`

Si un fichier `push.web.ts` existe à côté de `push.ts` , la version navigateur utilise le `.web.ts` (car le trousseau sécurisé ou le push natif n'existent pas dans un navigateur).

13. La qualité : tests, lint, formatage, types

Avant chaque fin de tâche, **4 vérifications** : `pnpm lint` , `pnpm typecheck` , `pnpm test` , `pnpm format:check` .

Outil	Rôle	Où
Vitest	Lance les tests (« ce calcul donne-t-il le bon résultat ? »)	<code>*.spec.ts</code> (API), <code>*.test.ts</code> (mobile)
Supertest	Envoie de vraies requêtes HTTP à l'API dans les tests de bout en bout (e2e)	<code>api/test/*.e2e-spec.ts</code>
@nestjs/testing	Démarre l'application NestJS complète pour les tests	idem
tsc (TypeScript)	Vérifie les types sans produire de fichiers (<code>--noEmit</code>)	<code>pnpm typecheck</code>
oxlint	Lint ultra-rapide de l'API (écrit en Rust). Exemple de règle : une promesse non attendue est une erreur (<code>no-floating-promises</code>)	<code>api/.oxlintrc.json</code>
ESLint	Lint du site et du mobile	<code>web/eslint.config.mjs</code> , <code>mobile/eslint.config.js</code>
Prettier	Formate le code automatiquement (guillemets simples, virgules finales)	<code>.prettierrc</code> à la racine
EditorConfig	Réglages communs de l'éditeur (2 espaces, fin de ligne LF, UTF-8)	<code>.editorconfig</code>

Deux sortes de tests :

- **Tests unitaires** : testent une fonction isolée, sans base de données. Exemple : `annonces.regles.spec.ts` vérifie que `heureFr` affiche « 11h30 ».
- **Tests de bout en bout (e2e)** : démarrent l'API avec un vrai PostgreSQL et un vrai Redis, créent une école de test et vérifient tout le parcours (ex. une libération anticipée envoie bien un SMS au bon tuteur).

Le test **sécurité** (`securite.e2e-spec.ts`) liste **toutes les routes** de l'API et échoue si une nouvelle route n'a pas de règle d'accès.

14. La mise en production : Docker, Caddy, GitHub Actions, EAS

14.1 Docker et Docker Compose

Docker emballe une application et tout ce dont elle a besoin dans une **image** ; on lance l'image dans un **conteneur** isolé. Ainsi l'application tourne pareil sur tous les serveurs.

- `api/Dockerfile` et `web/Dockerfile` : recettes **multi-étapes** (une étape « construction » qui compile, une étape « exécution » légère qui ne garde que le résultat, lancée avec l'utilisateur `node`, pas `root`).
- `docker-compose.yml` (développement) : lance seulement PostgreSQL et Redis.
- `docker-compose.prod.yml` (production) : lance **6 services** : `postgres`, `redis`, `api`, `web`, `caddy`, `sauvegarde`, avec des **volumes** (dossiers persistants : données, photos, certificats) et des **contrôles de santé** (healthcheck).

Commandes utiles sur le serveur (alias `suivi = docker compose -f docker-compose.prod.yml --env-file .env.production`) : `suivi ps`, `suivi logs -f api`, `suivi restart web`.

14.2 Caddy 2

Serveur web **frontal** : il obtient et renouvelle **automatiquement** le certificat HTTPS (Let's Encrypt), compresse les réponses, et envoie `/api/*` vers l'API et le reste vers le site (`deploy/Caddyfile`). On appelle cela un **reverse proxy**.

14.3 Sauvegardes chiffrées (GPG)

Le service `sauvegarde` (`deploy/sauvegarde`) fait chaque nuit un `pg_dump` de la base et une archive des photos, **chiffrés en AES-256 avec GPG** (phrase secrète `SAUVEGARDE_PHRASE`), gardés 14 jours.

14.4 GitHub Actions (intégration continue)

`.github/workflows/ci.yml` : à chaque `push` sur `main`, GitHub démarre une machine Linux avec PostgreSQL et Redis, installe tout, puis lance formatage, lint, types, tests unitaires, tests e2e, construction de l'API, du site, du bundle Android et des images Docker. Si une étape échoue, le commit est marqué en rouge.

14.5 EAS (Expo Application Services)

Service d'Expo qui **construit** les applications Android (APK / AAB) et iPhone dans le cloud et les envoie aux magasins. Configuration : `mobile/eas.json` (profils `development`, `preview`, `production`) et `mobile/app.config.ts`.

14.6 Le serveur (VPS)

Un **VPS** (serveur privé virtuel) OVH sous **Ubuntu Linux**, avec le pare-feu **ufw** (ports 22, 80, 443 seulement). Guide complet : `docs/DEPLOIEMENT.md`.

15. Git et GitHub

- **Git** enregistre l'historique de chaque modification (**commit**). Commandes de base : `git status`, `git add`, `git commit -m "message"`, `git push` (envoyer sur GitHub), `git pull` (récupérer), `git log` (historique).
- **GitHub** héberge le dépôt en ligne (`MOUSTO-HUB/Suivi_eleve`) et lance la CI.
- `.gitignore` liste les fichiers à ne **jamais** envoyer (`.env` , `node_modules` , client Prisma généré...).
- `.gitattributes` force les fins de ligne Linux (LF) même sous Windows.

16. Tableau récapitulatif des versions

Technologie	Version	Partie
Node.js	≥ 22	API, site
pnpm	12.9	tout
TypeScript	6 (API, mobile), 5 (site)	tout
NestJS	12	API
Prisma	7.10	API
PostgreSQL	17	données
Redis	8	files, limites
BullMQ	6	files d'envoi
Next.js	16.3	site
React	19.2	site, mobile
Tailwind CSS	4	site
React Native	0.86	mobile
Expo SDK	57	mobile
Vitest	4 (API), 5 (mobile)	tests
Docker / Compose	—	production
Caddy	2	production

17. Parcours d'apprentissage conseillé

Apprendre **dans cet ordre**, en ouvrant à chaque étape les fichiers du projet indiqués.

Étape	Apprendre	Pratiquer sur le projet
1	HTML, CSS de base	<code>web/public/hors-ligne.html</code>
2	JavaScript : variables, fonctions, objets, tableaux, <code>async / await</code> , <code>fetch</code>	<code>web/public/sw.js</code>

Étape	Apprendre	Pratiquer sur le projet
3	TypeScript : types, interfaces, génériques	<code>api/src/common/*.ts</code> (petits fichiers simples)
4	Git et le terminal	<code>git log</code> , <code>git show 21117a0</code>
5	Node.js et pnpm	<code>package.json</code> , <code>pnpm dev:api</code>
6	HTTP / API REST / JSON	appeler <code>GET /api/sante</code> dans le navigateur
7	SQL puis Prisma	<code>schema.prisma</code> , <code>pnpm db:studio</code>
8	NestJS : module → contrôleur → service → DTO	tout le dossier <code>api/src/annonces</code>
9	Sécurité : JWT, hachage, guards	<code>api/src/auth</code>
10	Tests avec Vitest	<code>annonces.regles.spec.ts</code> puis un e2e
11	React : composants, props, état, hooks	<code>web/components/choix-theme.tsx</code>
12	Next.js : App Router, composants serveur, Server Actions	<code>web/app/(admin)/annonces</code>
13	Tailwind CSS	<code>web/components/ui.tsx</code> , <code>globals.css</code>
14	React Native + Expo	<code>mobile/src/app/(parent)/(onglets)/index.tsx</code>
15	Redis / BullMQ	<code>api/src/notifications/envoi.service.ts</code>
16	Docker / Compose / Caddy	<code>docker-compose.prod.yml</code> , <code>deploy/</code>
17	CI GitHub Actions	<code>.github/workflows/ci.yml</code>

Ressources officielles gratuites (en anglais, souvent traduites) :

- MDN Web Docs (HTML, CSS, JavaScript) : developer.mozilla.org/fr
- TypeScript Handbook : typescriptlang.org/docs
- NestJS : docs.nestjs.com
- Prisma : prisma.io/docs
- React : fr.react.dev
- Next.js : nextjs.org/docs (et la copie locale `web/node_modules/next/dist/docs/`)
- Tailwind CSS : tailwindcss.com/docs
- Expo et React Native : docs.expo.dev, reactnative.dev
- BullMQ : docs.bullmq.io
- Docker : docs.docker.com

Conseil : pour chaque technologie, lisez d'abord la partie correspondante du document `2-Structure-et-code` , puis modifiez une petite chose (un texte, une couleur) en local, lancez `pnpm test` et observez.